

Django Channels summary

Okay, let's turn this whole Django Channels documentation summary into an easy-to-follow **story in Banglish** 🙌

The Story of Django Channels

Ekta time chilo, Django bole ekta web framework chilo je HTTP request handle korte pare — mane simple web page request aar response. Sob bhalo choltachhilo, kintu ekdin developers ra chinta korlo:

“Ami jodi real-time chat banate chai? Ba IoT device theke data stream korte chai? Tokhon ki korbo?”

Django toh sudhu HTTP jane — kintu real-time communication, jemon **WebSocket**, **chat protocol**, **live update**, e sob ta HTTP er baire.

Ekhanei aslo **Django Channels** — Django'r ekta **superpower add-on!** ⚡

What Django Channels Does

Channels Django'r ability ke extend kore — mane Django'r upor ekta notun layer toiri kore ja **WebSocket**, **IoT**, aar onnano real-time protocol** handle korte pare.

Eita **ASGI** (Asynchronous Server Gateway Interface) er upor base kore.

ASGI holo Python'r ekta specification — eta basically ekta “upgrade” WSGI er (jeita Django normally use kore).

WSGI = for HTTP only

ASGI = for HTTP + WebSocket + more

So, Channels er maddhome Django hoye gelo **multi-talented** framework — synchronous ba asynchronous duibhabei connection handle korte pare. 😎

Components of Django Channels

Django Channels asole alada-alada part niye toiri:

1. **Channels** → Django'r main integration layer.
2. **Daphne** → ekta ASGI server ja HTTP and WebSocket request receive kore.
3. **asgiref** → ASGI'r core library, base backbone.
4. **channels_redis** → optional Redis backend, ja message passing er jonne use hoy.

Eta holo pura team — sobai milae ekta real-time system ke support kore.

Topics and Structure

Channels er documentation ta ekta boro “journey” er moto — jekhane step by step bujhano hoy:

1. **Introduction** – Channels er basic concept.

2. **Consumers** – ei gula holo Django view'r moto, but WebSocket er jonne.
3. **Routing** – different protocol (HTTP, WebSocket, etc.) ke alada alada route korar system.
4. **Channel Layers** – ekta messaging system ja different process ke ekshathe connect kore.
5. **Authentication, Sessions, Security** – Django'r default system ke WebSocket e extend kore.
6. **Testing & Deployment** – kibhabe async test likhbe aar production e deploy korbe.

Tarpor aro advanced topic ase – **Workers, Background Tasks, Channel Groups**, etc.

Example Story Scene

Dhoro tumi ekta **chat app** banachho.

- User message pathalo frontend theke → WebSocket diye backend e gelo.
- Backend e **Consumer** message ta receive korlo.
- Tarpor message ta **Channel Layer** diye onno connected user der pathano holo.
- Sob real-time update holo — page reload chara! 🌟

Ekhon Django jeno **magic chat wizard** hoye gelo.

Developer Tips

Channels 4.x version holo latest, kintu purono version (1.x, 2.x, 3.x) er documentation o ache — niche version selector diye choose kora jay.

Ar jodi tumi contribute korte chao, Channels'r community khub friendly — beginners der jonno o help ase.

Summary in Short

- **Django Channels** extends Django beyond HTTP.
 - **Built on ASGI**, supports async communication.
 - Handles **WebSocket, chat, IoT**, etc.
 - Uses **Daphne, asgiref, channels_redis** as core parts.
 - Introduces **Consumers, Routing, Channel Layers**.
 - Perfect for **real-time apps** (chat, live dashboard, notifications).
-

Would you like me to make a **visual story map** (like a flowchart or comic-style diagram) of this Django Channels story? It'll help you remember the concept super easily.

Perfect 😎 — let's continue the **Banglish story mode summary** of this second part of the Django Channels documentation.

We'll keep it like a storytelling series — imagine Django Channels as a hero getting new powers 🔥

Chapter 2: The Journey of Django Channels (Introduction → Django

Integration)

Introduction: A New Power Awakens

Once upon a time, Django chilo ekta HTTP-only hero — jekhane sob kichu request-response model e base chilo.

But ekdin, “real-time connection” naam er ekta challenge elo — ja WebSocket, MQTT, chatbot, IoT connection sob chay ekta **long-running link** server aar client er moddhe.

So Django’r jonno eshe gelo **Channels** — ekta wrapper je Django ke asynchronous power diye upgrade korlo! ⚡

Now Django can handle not just **HTTP**, but also **WebSocket**, **MQTT**, **Chatbot**, even **amateur radio** (yes, seriously 😊).

Ar sab theke important holo — ei power add hoileo, Django’r simplicity thik thaklo.

You can still write:

- old-style **sync code** (like Django views),
- or fully **async code**,
- or even mix both.

Channels also integrates Django’s **Auth system**, **Session system**, and adds **Channel Layers** for process-to-process communication — mane ekta app’er ekta part theke onno part ke message pathano possible hoye gelo.

So, it’s like — Django now learned to talk **beyond HTTP** 💬.

Turtles All The Way Down

Channels er ekta motto ache — “Turtles All The Way Down.” 🐢

Mane holo, sob kichu ekta **ASGI application**.

Small consumer, large router, full Django app — sob ekshathe layer by layer ASGI'r upor base kore chole.

Even ekta simplest **consumer** (mini app) o independent vabe run korte pare.

👉 **ASGI** holo asynchronous server interface ja Django Channels'r backbone.

Eta WSGI'r moto, but async-friendly.

You can mix different frameworks and servers (e.g., Daphne, Uvicorn) freely — not locked in one.

So, Django HTTP + Channels WebSocket + Router = ekta boro “Unified Django Universe”. 🌌

🔍 **Scopes and Events: How Things Actually Work**

Think of every connection as a **storyline** — called a **scope**.

Inside that storyline, multiple **events** happen.

Example:

📄 **HTTP**

- User sends HTTP request
- A **scope** is created (with path, method, headers)
- A **http.request** event is fired
- Django/Channels process it → returns a **http.response**
- Scope destroyed ✅

Chatbot

- User sends first message
- A **chat scope** opens (with user info)
- Event: **chat.received_message**
- App can respond with **chat.send_message**
- After session ends → scope closes.

So in short —

 **Scope = connection context**

 **Events = actions happening during that connection**

Ar ei events handle korar jonno Channels use kore **Consumers**.

What is a Consumer?

A **Consumer** holo Channels'r “view function” — but smarter and long-living.

When a new WebSocket or request ashe → Channels **routes** it to the correct **Consumer** and runs an instance of it.

So ekta consumer holo mini-app je:

- **connect** kore client er sathe
- **receive** kore message
- **disconnect** hole cleanup kore

Example:

python

```
class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.username = "Anonymous"
        self.accept()
        self.send(text_data="[Welcome %s!]" % self.username)

    def receive(self, *, text_data):
        if text_data.startswith("/name"):
            self.username = text_data[5:].strip()
            self.send(text_data="[set your username to %s]" %
self.username)
        else:
            self.send(text_data=self.username + ": " +
text_data)
```

Eta synchronous style — easy and Django-like.

But jodi async korte chao:

python

```
class PingConsumer(AsyncConsumer):
    async def websocket_connect(self, message):
        await self.send({"type": "websocket.accept"})

    async def websocket_receive(self, message):
        await asyncio.sleep(1)
        await self.send({
            "type": "websocket.send",
            "text": "pong",
        })
```

Ei duita style e possible — Django stays flexible 😊.



Routing and Multiple Protocols

Django Channels er ekta **super router** ache je ek shathe multiple consumer combine korte pare.


```
python
```

```
application = URLRouter([
    path("chat/admin/", AdminChatConsumer.as_asgi()),
    path("chat/", PublicChatConsumer.as_asgi()),
])
```

But the fun part?

You're not limited to HTTP or WebSocket only 😎

You can add **Telegram**, **MQTT**, **IoT**, etc.

Example:

```
python
```

```
class ChattyBotConsumer(SyncConsumer):
    def telegram_message(self, message):
        self.send({
            "type": "telegram.message",
            "text": "You said: %s" % message["text"],
        })
```

And then route both:

```
python
```

```
application = ProtocolTypeRouter({
    "websocket": URLRouter([...]),
    "telegram": ChattyBotConsumer.as_asgi(),
})
```

Now your Django project can talk to **browser users**, **telegram bots**, or even **IoT devices** — ekdom ek platform e.

Cross-Process Communication

Dhoro tumi ekta chatroom banachho.

User1 ekta message pathalo, but onno users alada server process e connected.

So, tumi chaicho jeno sobai message ta pay — but kibhabe?

Channels e ache **Channel Layers** — ja ekta message passing system.

Eta ekdom “communication bridge” moto kaj kore.

Example:

```
python

self.channel_layer.send(
    'event',
    {
        'type': 'message',
        'channel': channel,
        'text': text,
    }
)
```

Ar kono specific worker ke message dite chaile:

```
python

self.channel_layer.send(
    "myproject.thumbnail_notifications",
    {
        "type": "thumbnail.generate",
        "id": 90902949,
    }
)
```

Eta Redis backend diye run korte parbe — ja async communication ke super fast kore. ⚡

Django Integration: Familiar Yet Powerful

Channels Django'r existing systems er sathe **perfectly integrated**. Mane sessions, authentication, middleware — sob use korte parbe as-is.

Example setup:

```
python

from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from channels.security.websocket import AllowedHostsOriginValidator
from django.core.asgi import get_asgi_application
from django.urls import re_path

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter([
                re_path(r"^front(end)/$",
consumers.AsyncChatConsumer.as_asgi()),
            ])
        ),
    ),
})
```

So ekhane:

- **HTTP** handled by Django as usual
- **WebSocket** handled by Channels
- **AuthMiddlewareStack** diye user authentication easily manage hoy
- **AllowedHostsOriginValidator** ensures security

Boom 🌟 — now Django is async, real-time, and secure — without losing its simplicity.

🚩 Summary of This Chapter

Concept	Summary
Channels	Extends Django beyond HTTP
ASGI	Async server system replacing WSGI
Scope	Connection context
Event	Actions inside a scope
Consumer	Long-lived async/sync mini-app
Routing	Combine multiple protocols
Channel Layers	Communicate between processes
Integration	Works seamlessly with Django's auth, session, middleware

Would you like me to make the **next part (Consumers + Channel Layers + Deployment)** in the same Banglish storytelling style — like Chapter 3 of this Django adventure series?

I can make it feel like a “developer fantasy novel” 📖 ✨

Perfect 🔥! Time for **Chapter 3** of our Django Channels story in **Banglish style** — this time, we're learning how to *summon* Channels into our Django world 🧙 ✨



Chapter 3: The Installation – Summoning the Power of Channels

So far, Django Channels has become a superhero in our story — but ekhon she ke amader project e niye ashte hobe.

Let's see how to **install and activate** this new real-time magic ☐



Step 1: The Summoning Spell (Installation Command)

Channels already lives in **PyPI**, the great library of Python spells 📜

To summon it, open your terminal and chant this:

```
bash
```

```
python -m pip install -U 'channels[daphne]'
```

✨ Boom! This installs two powerful tools:

1. **Channels** – the real-time extension of Django
2. **Daphne** – the ASGI server that runs Django Channels apps

If you already have another ASGI server (like Uvicorn), you can just install plain Channels:

```
bash
```

```
pip install channels
```

So basically:

- 'channels[daphne]' → full setup
- 'channels' → just the core

Step 2: Adding Daphne to Django's Army (INSTALLED_APPS)

Now that Daphne has joined your project, Django must recognize it. Go to your **settings.py** and at the top of **INSTALLED_APPS**, add:

```
python

INSTALLED_APPS = [
    "daphne", # 👉 must be first!
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.sites",
    ...
]
```

Why at the top? 

Because Daphne overwrites Django's **runserver** command with an **ASGI-compatible** version — so Django can now handle **async connections**.

If you also want to use Channels' **worker system**, you can add this too:

```
python

"channels"
```

Now you'll unlock commands like:

```
bash

python manage.py runworker
```

for background event workers 💪

Step 3: Setting Up the Magic Gateway (asgi.py)

Next, go to your project directory — usually something like:

```
bash

myproject/asgi.py
```

Replace or edit it like this:

```
python

import os
from channels.routing import ProtocolTypeRouter
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
    'mysite.settings')

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    # Just HTTP for now – more protocols like WebSocket can
    # come later!
})
```

This file is basically your **ASGI gateway**,
where Django's HTTP world and Channels' async world meet. 🌍

📄 Step 4: The Final Binding (settings.py)

Now, tell Django that Channels' new router is the **main entry point**.

Add this line in your **settings.py**:

```
python

ASGI_APPLICATION = "myproject.asgi.application"
```

That's your **root router** —
all async traffic (like WebSocket, chat, etc.) will flow through this
"magic portal." 🌐

🧠 Step 5: Run the Server!

Now you can simply run:

```
bash

python manage.py runserver
```

And guess what? 🙄

It's not the old **runserver** anymore —
it's **Daphne-powered runserver**,
which means your project can now handle **WebSockets and async protocols** side-by-side with HTTP.

Your Django app just evolved into **Django Real-Time Mode** ⚡

⚠️ Note from the Master Scrolls (Common Conflicts)

Be careful  — some other Django apps may also modify `runserver`. For example:

`whitenoise.runserver_nostatic` can **conflict** with Daphne's version.

Fix:

- Always keep "daphne" at the **top** of your `INSTALLED_APPS`, or
- Remove the conflicting app if not needed.

Simple, but saves you hours of headache 

Step 6: Installing the Latest Developer Version (Optional)

If you're the type of developer who likes to live on the *bleeding edge*



you can install the latest dev version directly from GitHub:

```
bash

git clone git@github.com:django/channels.git
cd channels
<activate your virtualenv>
pip install -e .
```

The `-e .` means "install from this local folder in editable mode."

Now you can test the newest features before anyone else 

Summary – Installation in Story Form

Step	What You Did	Why
1	Installed <code>channels[daphne]</code>	Adds Channels + Daphne server
2	Added " <code>daphne</code> " to <code>INSTALLED_APPS</code>	Enables async runserver
3	Created <code>asgi.py</code> with <code>ProtocolTypeRouter</code>	Sets up entry point
4	Set <code>ASGI_APPLICATION</code>	Tells Django which app to load
5	Handled runserver conflicts	Keeps server stable
6	(Optional) Installed dev version	Try latest features

🎉 Congratulations!

Your Django project now officially speaks **ASGI** — ready for real-time magic, WebSockets, chat, IoT, and more 🌟

Would you like me to continue with **Tutorial Part 1: Basic Setup** next (in the same Banglish story mode)?

That's where we actually *start coding* a real **chat app** step-by-step 💬

Nice — ekhon **Tutorial Part 1: Basic Setup** ta Banglish story-mode e diye dichhi, step-by-step, like a mini developer adventure. Ready? 🚀

👤 Chapter: Tutorial Part 1 — Basic Setup (Banglish Story Mode)

📢 Prologue: Ki banabo

Tumi ekta simple **chat server** banate jachcho. Dui ta page thakbe:

- **Index page** — user room-name type kore join korbhe.
- **Room page** — oi room-er messages dekhabe, WebSocket diye realtime communicate korbhe.

Assumption: tumi already Django, Channels, Daphne install korecho (na korle ager Installation chapter follow kor).

✓ Check your tools

Terminal e check korte paro:

- Django version:
`python3 -m django --version`
- Channels + Daphne installed kina:
`python3 -c 'import channels; import daphne; print(channels.__version__, daphne.__version__)'`

Tutorial ta **Channels 4.0** er jonno — Python 3.7+ and Django 3.2+ compatible.

Redis (optional but later required)

Later parts e Channel Layer (Redis) use korbhe — easiest bhabe Docker diye Redis run korte bola ache. For now index view run korte direct **runserver** cholbe, kintu Redis lagle Docker install koro.

Step 1 — Project create kora

Jodi project na thake, create koro:

```
bash

django-admin startproject mysite
```

Folder structure:

```
markdown

mysite/
  manage.py
  mysite/
    asgi.py
    settings.py
    urls.py
    wsgi.py
```

Step 2 — Chat app create koro

Project root (jekhane `manage.py`) e:

```
bash

python3 manage.py startapp chat
```

Default files asbe — tutorial e sudhu `chat/views.py` and `chat/__init__.py` lage bole baki gulo remove korte bola ache (tutorial simple rakhte).

App install korte `mysite/settings.py` e add koro:

```
python

INSTALLED_APPS = [
    'chat',
```

```
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
]
```



Step 3 — Index view + template

chat/templates/chat/index.html file banao, content:

html

```
<!-- chat/templates/chat/index.html -->
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"/><title>Chat Rooms</title></head>
<body>
  What chat room would you like to enter?<br>
  <input id="room-name-input" type="text" size="100"><br>
  <input id="room-name-submit" type="button" value="Enter">
  <script>
    document.querySelector('#room-name-input').focus();
    document.querySelector('#room-name-input').onkeyup =
function(e) {
  if (e.key === 'Enter') {
document.querySelector('#room-name-submit').click(); }
  };
    document.querySelector('#room-name-submit').onclick =
function(e) {
  var roomName = document.querySelector('#room-name-
input').value;
  window.location.pathname = '/chat/' + roomName +
  '/';
  };
  </script>
</body>
</html>
```

chat/views.py e index view:

```
python
```

```
from django.shortcuts import render

def index(request):
    return render(request, "chat/index.html")
```

chat/urls.py create koro:

```
python
```

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name="index"),
]
```

Project root **mysite/urls.py** e include koro:

```
python
```

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("chat/", include("chat.urls")),
    path("admin/", admin.site.urls),
]
```

Run server:

```
bash
```

```
python3 manage.py runserver
```

Open **http://127.0.0.1:8000/chat/** — index page dekhite pabe. Type “lobby” -> **/chat/lobby/** e jabe (room view ekhono nai, tai 404 pabe).

Stop server with CTRL-C.

Step 4 — Integrate Channels (Django + ASGI)

Ekhon Channels use korbo — asgi routing add korte `mysite/asgi.py` update koro:

```
python

# mysite/asgi.py
import os
from channels.routing import ProtocolTypeRouter
from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"mysite.settings")

application = ProtocolTypeRouter({
    "http": get_asgi_application(),
    # later add websocket etc.
})
```

`settings.py` e `daphne` add koro (top e rakhbe bhalo):

```
python

INSTALLED_APPS = [
    'daphne',
    'chat',
    ...
]
```

aro add koro:

```
python

# settings.py
ASGI_APPLICATION = "mysite.asgi.application"
```

Ekhon `runserver` dile Dekhbe **Starting ASGI/Daphne** ... — mane Daphne runserver dhore niyeche. Run kore abar `http://127.0.0.1:8000/chat/` dekho — index view thik ache.

Note: jodi onnanno third-party app `runserver` override kore, conflict hote pare — solution: make sure '`daphne`' top e thakbe or remove conflicting app.

Next step

Tutorial Part 1 finish — server ASGI-ready holo and index view kaj kore. Tutorial 2 e WebSocket room view, routing, consumers, and Redis channel layer er kotha ashbe.

Short Summary (Quick Cheatsheet)

- Create project + `chat` app.
 - Add index template + view + url.
 - Test with `runserver` → see index page.
 - Integrate Channels: update `asgi.py`, add '`daphne`' to `INSTALLED_APPS`, set `ASGI_APPLICATION`.
 - Run server → Daphne dev server starts.
 - Next: implement room view + WebSockets (Tutorial 2).
-

Chao ami same style e **Tutorial Part 2** o diye di, jekhane room view, WebSocket consumer, routing, ebong Redis channel layer diye chat purota realtime banabo?

Nice — Tutorial Part 2 ta Banglish story-mode e diye dichhi, step-by-step, ja diye tumi ekta **realtime multi-tab chat** banaite parba.

Ready? 🚀



Chapter: Tutorial Part 2 — Implement a Chat Server (Banglish Story Mode)



Objective

Part 1 e index page banailam. Ebar room page banabo — jekhane WebSocket diye messages pathabe, ar same room e thaka onno browsers o pabe (i.e., broadcast).



Step A — Room template & view

File: chat/templates/chat/room.html

html

```
<!-- chat/templates/chat/room.html -->
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"/><title>Chat Room</title></head>
<body>
  <textarea id="chat-log" cols="100" rows="20"></textarea><br>
  <input id="chat-message-input" type="text" size="100"><br>
  <input id="chat-message-submit" type="button" value="Send">
  {{ room_name|json_script:"room-name" }}
  <script>
    const roomName = JSON.parse(document.getElementById('room-
name').textContent);
    const chatSocket = new WebSocket(
      'ws://' + window.location.host + '/ws/chat/' + roomName
+ '/'
    );
```

```

chatSocket.onmessage = function(e) {
    const data = JSON.parse(e.data);
    document.querySelector('#chat-log').value +=
(data.message + '\n');
};
chatSocket.onclose = function(e) { console.error('Chat
socket closed unexpectedly'); };

    document.querySelector('#chat-message-input').focus();
    document.querySelector('#chat-message-input').onkeyup =
function(e) {
        if (e.key === 'Enter') document.querySelector('#chat-
message-submit').click();
    };
    document.querySelector('#chat-message-submit').onclick =
function(e) {
        const messageInputDom = document.querySelector('#chat-
message-input');
        const message = messageInputDom.value;
        chatSocket.send(JSON.stringify({'message': message}));
        messageInputDom.value = '';
    };
</script>
</body>
</html>

```

View: chat/views.py (add room view)

```

python

from django.shortcuts import render

def index(request):
    return render(request, "chat/index.html")

def room(request, room_name):
    return render(request, "chat/room.html", {"room_name":
room_name})

```

URL: chat/urls.py

```
python
```

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name="index"),
    path("<str:room_name>/", views.room, name="room"),
]
```

Run server and open <http://127.0.0.1:8000/chat/lobby/> — UI ready but WebSocket will 500 until consumer exists.

Step B — First consumer (echo)

Create `chat/consumers.py`:

```
python
```

```
import json
from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.accept()

    def disconnect(self, close_code):
        pass

    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json["message"]
        self.send(text_data=json.dumps({"message": message}))
```

This is a simple **synchronous** WebsocketConsumer that just echoes back messages to the same client.



Step C — WebSocket routing for chat app

Create `chat/routing.py`:

```
python

from django.urls import re_path
from . import consumers

websocket_urlpatterns = [
    re_path(r"ws/chat/(?P<room_name>\w+)/$",
        consumers.ChatConsumer.as_asgi()),
]
```

(we use `re_path` for `URLRouter` compatibility)



Step D — Hook routes into project ASGI

Edit `mysite/asgi.py` to include websocket routing, auth middleware, and allowed-hosts validator:

```
python

import os
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import AllowedHostsOriginValidator
from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
    "mysite.settings")
django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
```

```

        URLRouter(websocket_urlpatterns)
    ),
}

```

This makes Channels route `ws://` connections to your consumer, and populates `scope["user"]` via `AuthMiddlewareStack`.

Step E — Test echo consumer

Run migrations (sessions/auth need DB):

```

bash

python3 manage.py migrate
python3 manage.py runserver

```

Open two tabs to `/chat/lobby/`. With echo consumer you'll see your typed message echoed back in the tab that sent it — but it won't broadcast to other tabs (yet).

Step F — Enable cross-process communication: Channel layer (Redis)

To broadcast, consumers must talk to each other. Channels uses a **channel layer** (Redis backend common).

1. Start Redis (recommended via Docker):

```

bash

docker run --rm -p 6379:6379 redis:7

```

2. Install Redis backend for Channels:

```
bash

python3 -m pip install channels_redis
```

3. Configure `settings.py`:

```
python

ASGI_APPLICATION = "mysite.asgi.application"

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("127.0.0.1", 6379)],
        },
    },
}
```

4. Quick shell test:

```
python

$ python3 manage.py shell
>>> import channels.layers
>>> channel_layer = channels.layers.get_channel_layer()
>>> from asgiref.sync import async_to_sync
>>> async_to_sync(channel_layer.send)('test_channel', {'type':
'hello'})
>>> async_to_sync(channel_layer.receive)('test_channel')
{'type': 'hello'}
```

If that returns the message, channel layer <-> Redis is working.

Step G — Update consumer to use groups (broadcast)

Replace `chat/consumers.py` with:

python

```
import json
from asgiref.sync import async_to_sync
from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.room_name = self.scope["url_route"]["kwargs"]
["room_name"]
        self.room_group_name = f"chat_{self.room_name}"

        # Join room group
        async_to_sync(self.channel_layer.group_add)(
            self.room_group_name, self.channel_name
        )

        self.accept()

    def disconnect(self, close_code):
        # Leave room group
        async_to_sync(self.channel_layer.group_discard)(
            self.room_group_name, self.channel_name
        )

    # Receive message from WebSocket
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json["message"]

        # Send message to room group
        async_to_sync(self.channel_layer.group_send)(
            self.room_group_name,
            {"type": "chat.message", "message": message}
        )

    # Receive message from room group
    def chat_message(self, event):
        message = event["message"]
```

```
# Send message to WebSocket
self.send(text_data=json.dumps({"message": message}))
```

Key notes:

- `self.scope["url_route"]["kwargs"]["room_name"]` gets `room_name` from URL.
- `room_group_name` constructed as `chat_<room_name>` — group names must be ASCII alphanumeric, hyphens, underscores, or periods (no spaces/special chars), and limited length (Redis backend default ~100).
- Channel layer methods (`group_add`, `group_send`, etc.) are **async**, so we wrap them with `async_to_sync` because our consumer is synchronous. (Alternatively write an `AsyncConsumer`.)
- `group_send` event has `"type": "chat.message"` → Channels maps that to `chat_message` method on each consumer (dots → underscores).

✅ Step H — Final test (broadcast works)

1. Make sure Redis is running.
2. `python3 manage.py runserver`
3. Open two browser tabs at `http://127.0.0.1:8000/chat/lobby/`.
4. Type message in tab A → both tab A and tab B should show the message.

Congrats — you now have a **working multi-client chat room** using Django Channels + Redis!

⚠ Caveats & tips

- Group names must be sanitized: only [A-Za-z0-9-_.], limited length.
- If you prefer async, write `AsyncConsumer` / `AsyncWebsocketConsumer` and use `await` instead of `async_to_sync`.
- Remember to stop Redis (Ctrl-C) when done if using `docker run --rm`.
- For production, consider authentication, origin checks, TLS (wss://), and proper Redis/ASGI server setup.

🚩 Summary (one-liner)

Part 2 adds the room template + view, creates a WebSocket consumer, routes WS paths, and uses a Redis-backed channel layer so multiple consumer instances in the same room can broadcast messages to each other.

Chao ami **Tutorial Part 3** o Banglish story-mode e likhe di— ebar worker/background tasks, testing, deployment, and some security best-practices diye?

Sure! Ebar asche **Tutorial Part 3 – Rewrite Chat Server as Asynchronous**, abr Banglish story mode e, ja tumi easily bujhte aar explain korte parba! 😎

🚀 Chapter: Tutorial Part 3 — Chat Server ke Fully Asynchronous Korbo

Recap

Part 2 e amra **WebSocket consumer create korlam**, Redis channel layer add kore **multi-client chat** kaj koraitesi.

But ekhon amar ChatConsumer synchronous (**WebsocketConsumer**), mane thread use kore I/O handle kore.

Threads = heavy resource consumption.

● Solution → Convert to **async-based consumer**

Async code **faster, scalable & efficient** — especially WebSocket er moto long-lived connection e.

Why Async?

Sync Consumer	Async Consumer
def use kore	async def use kore
async_to_sync() lagto	direct await call kora jay
Thread create hoy (resource-heavy)	Event-loop e run hoy (lightweight)
Easy to write	Better performance

Our ChatConsumer doesn't use Django ORM or blocking code → so fully async e rewrite kora safe.

Step: Rewrite Consumer as Async

 `chat/consumers.py` replace kore eta use koro:

python

```
# chat/consumers.py
import json
from channels.generic.websocket import AsyncWebsocketConsumer

class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.room_name = self.scope["url_route"]["kwargs"]
        ["room_name"]
        self.room_group_name = f"chat_{self.room_name}"

        # Join room group
        await
self.channel_layer.group_add(self.room_group_name,
self.channel_name)

        await self.accept()

    async def disconnect(self, close_code):
        # Leave room group
        await
self.channel_layer.group_discard(self.room_group_name,
self.channel_name)

    async def receive(self, text_data): # Receive WebSocket
        text_data_json = json.loads(text_data)
        message = text_data_json["message"]

        # Broadcast message to group
        await self.channel_layer.group_send(
            self.room_group_name,
            {"type": "chat.message", "message": message}
        )

    async def chat_message(self, event): # Receive from group
        message = event["message"]
        await self.send(text_data=json.dumps({"message":
message}))
```

What Changed?

Old	New
<code>WebsocketConsumer</code>	<code>AsyncWebsocketConsumer</code>
<code>def connect(self):</code>	<code>async def connect(self):</code>
<code>async_to_sync(method)()</code>	<code>await method()</code>
More overhead	Better optimized

🔥 No need for `async_to_sync()` anymore!



Test Koro (Same As Before)

Run:

```
bash

python3 manage.py runserver
```

Open **2 ta tab**:

```
ruby

http://127.0.0.1:8000/chat/lobby/
```

Message type koro → 🔥 **dui tab e real-time echo hobe**

🎉 Congrats! Tumi ekhon **fully asynchronous chat server** implement kore felecho.



Advanced Note (Optional)

Jodi async consumer Django models use kore (ORM), tokhon blocking code async e call korte parbe:

python

```
from channels.db import database_sync_to_async

@database_sync_to_async
def save_message(self, text):
    Message.objects.create(msg=text)
```

But performance kombe — best practice holo async-native code use kora.

Final Summary (Brain-Friendly)

vbnet

Sync → add threads → slow
 Async → run on event loop → fast 🚀

Old: WebsocketConsumer + async_to_sync
 New: AsyncWebsocketConsumer + await only

Ebar tumi likhte paro je:

"My chat application is now FULLY ASYNCHRONOUS using AsyncWebsocketConsumer!"

📌 Next Chapter (Tutorial 4) chao?

👉 Part 4 e amra **Automated Testing** + **better architecture** korebo — chai story mode e continue kori? 😊

Sure! Eibar asche **Tutorial Part 4 – Automated Testing**, abar Banglish story mode e 🤖 — for ensuring tomar chat server future teo thik moto kaj korbe.

Chapter: Tutorial Part 4 — Automated Testing (Banglish Story Mode)

Goal

Amra already fully **asynchronous, real-time chat server** banai felechi (Tutorial 3).

Ebar kori **automated testing**, jate **future development e bug dhore pawa jay automatically**.

We'll use:

- ✓ **Selenium** → browser automation (Chrome)
- ✓ **ChannelsLiveServerTestCase** → ASGI + WebSocket test support

What will be tested?

1 Jodi ekta room e message dei → same room e thaka sobai message ta dekhbe

2 Jodi different room e dei → onno room e message dekha jabe na
👉 exactly how a real chat app should behave!

Step 1 — Install required tools

Make sure **Chrome browser** installed. Then:

```
bash
```

```
python3 -m pip install selenium
```

Also ensure **chromedriver** installed & PATH e ache.



Step 2 — Create the test file

 chat/tests.py

python

```
from channels.testing import ChannelsLiveServerTestCase
from selenium import webdriver
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.wait import WebDriverWait

class ChatTests(ChannelsLiveServerTestCase):
    serve_static = True # Acts like StaticLiveServerTestCase

    @classmethod
    def setUpClass(cls):
        super().setUpClass()
        try:
            cls.driver = webdriver.Chrome() # Requires
chromedriver
        except:
            super().tearDownClass()
            raise

    @classmethod
    def tearDownClass(cls):
        cls.driver.quit()
        super().tearDownClass()

    def
test_when_chat_message_posted_then_seen_by_everyone_in_same_ro
om(self):
    try:
        self._enter_chat_room("room_1")
        self._open_new_window()
        self._enter_chat_room("room_1")

        self._switch_to_window(0)
        self._post_message("hello")
```

```

        WebDriverWait(self.driver, 2).until(lambda _:
"hello" in self._chat_log_value)

        self._switch_to_window(1)
        WebDriverWait(self.driver, 2).until(lambda _:
"hello" in self._chat_log_value)
    finally:
        self._close_all_new_windows()

def
test_when_chat_message_posted_then_not_seen_by_anyone_in_diffe
rent_room(self):
    try:
        self._enter_chat_room("room_1")
        self._open_new_window()
        self._enter_chat_room("room_2")

        self._switch_to_window(0)
        self._post_message("hello")
        WebDriverWait(self.driver, 2).until(lambda _:
"hello" in self._chat_log_value)

        self._switch_to_window(1)
        self._post_message("world")
        WebDriverWait(self.driver, 2).until(lambda _:
"world" in self._chat_log_value)
        self.assertTrue("hello" not in
self._chat_log_value)
    finally:
        self._close_all_new_windows()

# === Utility methods ===

def _enter_chat_room(self, room_name):
    self.driver.get(self.live_server_url + "/chat/")
    ActionChains(self.driver).send_keys(room_name,
Keys.ENTER).perform()
    WebDriverWait(self.driver, 2).until(lambda _:
room_name in self.driver.current_url)

def _open_new_window(self):
    self.driver.execute_script('window.open("about:blank",
"_blank");')
    self._switch_to_window(-1)

def _close_all_new_windows(self):
    while len(self.driver.window_handles) > 1:
        self._switch_to_window(-1)
        self.driver.execute_script("window.close();")

```



```

        self._switch_to_window(0)

    def _switch_to_window(self, window_index):
        self.driver.switch_to.window(self.driver.window_handles[window_index])

    def _post_message(self, message):
        ActionChains(self.driver).send_keys(message,
        Keys.ENTER).perform()

    @property
    def _chat_log_value(self):
        return self.driver.find_element(by=By.CSS_SELECTOR,
        value="#chat-log").get_property("value")

```

Why ChannelsLiveServerTestCase?

Django default test suite HTTP test kore (WSGI-based).

But amader chat app = ASGI + WebSocket

So use korte hobe —

✓ **ChannelsLiveServerTestCase** → properly handles WebSocket URLs like `/ws/chat/lobby/`

Step 3 — SQLite setup for test

SQLite by default in-memory mode e thake test e → Channels er jonno properly kaj kore na.

So settings.py update koro:

```

python

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",

```

```
"NAME": BASE_DIR / "db.sqlite3",
"TEST": {
    "NAME": BASE_DIR / "db.sqlite3",
},
},
}
```

Step 4 — Run the test!

```
bash
```

```
python3 manage.py test chat.tests
```

 Expected output:

```
bash
```

```
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
```

```
..
```

```
-----
```

```
Ran 2 tests in 5.014s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

 CONGRATULATIONS!

Tested + verified real-time chat server successfully.

Final Summary (Quick Revision)

Step	What You Did
	Installed Selenium & Chromedriver

Step	What You Did
2	Wrote end-to-end tests using browser automation
3	Used ChannelsLiveServerTestCase
4	Fixed SQLite settings
5	Ran tests and verified correct WebSocket behavior



End of the Tutorial Series!

You have now successfully:

- ✓ Created a Django + Channels chat server
- ✓ Converted to async for performance
- ✓ Used Redis channel layer for broadcasting
- ✓ Written automated tests using Selenium

🔥 You're now ready to build **production-ready real-time apps** using Django Channels!

Would you like next:

- 👉 Deployment and scaling guide for Channels
- 👉 Authentication & user-specific chat
- 👉 Or converting this into a **multi-room real-time chat UIs** using Tailwind + JS?

Let me know — ami guide kore debo next level e 😎